EXAMEN ÉCRIT – DÉVELOPPEMENT MOBILE (BTS)

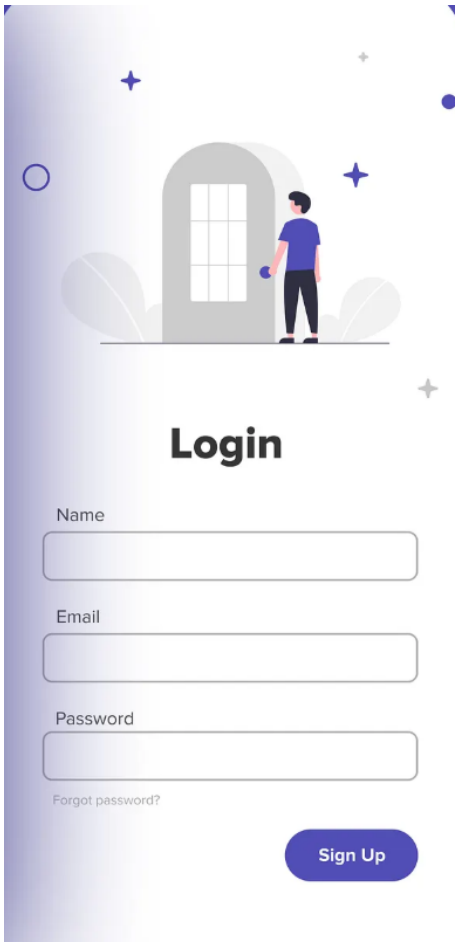
Programmation Android — Durée : 2h

Consignes :
- Examen écrit (aucun IDE autorisé)
- Compléter le code et répondre aux questions
- Les parties manquantes sont indiquées par

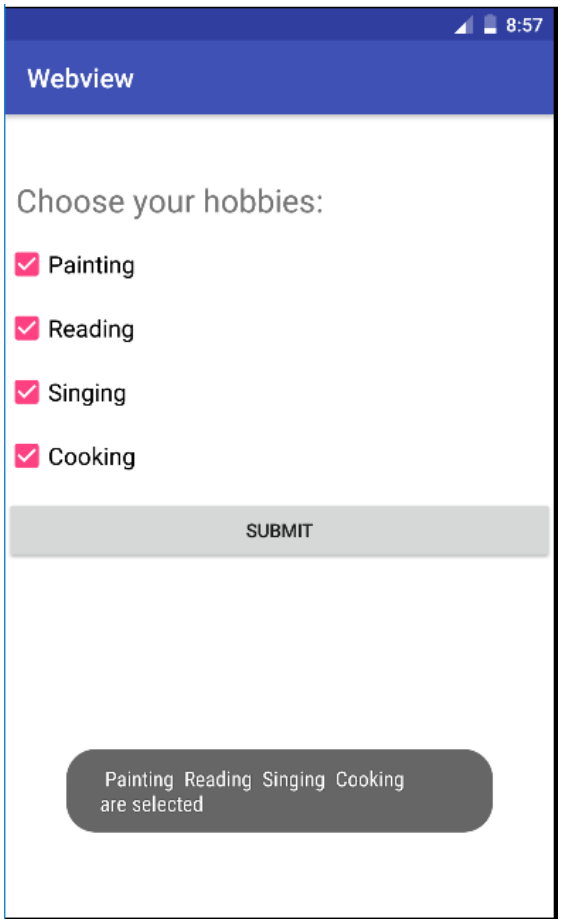
Contexte général

Vous développez une application Android appelée **SmartAccess**. L'application contient deux activités :
- **LoginActivity** : écran de connexion avec illustration
- **DashboardActivity** : écran permettant à l'utilisateur de sélectionner une option via des **CheckBox** puis de valider son choix

Les interfaces correspondantes sont fournies avec le sujet.



(a) Écran de connexion (LoginActivity)



(b) Écran de sélection avec CheckBox

FIGURE 1 – Interfaces de l'application SmartAccess

PARTIE 1 — AndroidManifest.xml (4 pts)

Compléter le fichier `AndroidManifest.xml` afin que :
— `LoginActivity` soit l'activité de démarrage de l'application
— `DashboardActivity` soit une activité interne accessible uniquement depuis l'application

```
<application
    android:allowBackup="true"
    android:theme="@style/Theme.App">

    <activity
        android:name=".DashboardActivity"
        android:exported="__________">
        <!-- activité secondaire -->
    </activity>

    <activity
        android:name=".LoginActivity"
        android:exported="__________">
        <intent-filter>
            <action android:name="__________" />
            <category android:name="__________" />
        </intent-filter>
    </activity>

</application>
```

Questions

1. Quelle activité sera lancée au démarrage de l'application ? Justifier.
2. À quoi servent les balises `<action>` et `<category>` dans un `intent-filter` ?
3. Que signifie l'attribut `android:exported` ?
4. Pourquoi `DashboardActivity` doit-elle avoir une valeur différente de `exported` par rapport à `LoginActivity` ?

PARTIE 2 — Interface Login (6 pts)

L'interface Login est inspirée d'un écran de connexion mobile réel. Elle contient :
— une image illustrative ,
— deux champs de saisie,
— un bouton de connexion,
— un fond coloré.

2.1 Layout XML — ConstraintLayout

Compléter le layout suivant :

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="__________"
```

```
    android:layout_height="__________"
    android:background="@color/__________">

    <ImageView
        android:id="@+id/imgAvatar"
        android:layout_width="__________"
        android:layout_height="__________"
        android:src="@drawable/__________"
        app:__________="__________"
        app:__________="__________"
        app:__________="__________" />

    <EditText
        android:id="@+id/etLogin"
        android:layout_width="__________"
        android:layout_height="wrap_content"
        android:hint="Name"
        app:__________="__________"
        app:__________="__________"
        app:__________="__________" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="__________"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="__________"
        app:__________="__________"
        app:__________="__________"
        app:__________="__________" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sign Up"
        app:__________="__________"
        app:__________="__________"
  />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Questions

1. Quelle différence entre `wrap_content` et `match_parent` ?
2. Pourquoi utilise-t-on `ConstraintLayout` dans cette interface ?
3. Dans quel dossier se trouve l'image utilisée par `ImageView` ?
4. Où est définie la couleur utilisée comme background ?

PARTIE 3 — LoginActivity (6 pts)

3.1 Méthode de création

Compléter la méthode suivante (nom de la fonction à deviner) :

```
public class LoginActivity extends AppCompatActivity {

    EditText etLogin, etPassword;
    Button btnLogin;

    @Override
    protected void __________(Bundle savedInstanceState) {
        super.__________(savedInstanceState);

        ________________________________;

        etLogin = ______________________;
        etPassword = ___________________;
        btnLogin = ______________________;
    }
}
```

3.2 Gestion du bouton Connexion

Objectif :
- Vérifier que les champs ne sont pas vides
- Afficher un message d'erreur si nécessaire
- Sinon, ouvrir `DashboardActivity` et envoyer le login

```
btnLogin.setOnClickListener(v -> {

    String login = ____________________;
    String pwd   = ____________________;

    if (login.isEmpty() || pwd.isEmpty()) {
        ________________________________;
    } else {

        Intent i = new Intent(
            LoginActivity.this,
            __________________________
        );

        i.putExtra("login", ____________);

        ________________________________;
    }
});
```

PARTIE 4 — DashboardActivity (6 pts)

4.1 Récupération des données

```
String login = getIntent().____________________;
```

4.2 Gestion d'une CheckBox

Objectif :
- Si la CheckBox est cochée : afficher le message **"Notifications activées"**
- Sinon : afficher le message **"Notifications désactivées"**

```
cbNotif.setOnCheckedChangeListener(
    (buttonView, isChecked) -> {
        if (isChecked) {
            ______________________;
        } else {
            ______________________;
        }
    }
);
```

4.3 Bouton SUBMIT

Objectif : Afficher un message résumant le choix de l'utilisateur lors du clic sur le bouton.

```
btnSubmit.setOnClickListener(v -> {
    // Afficher un message indiquant que le choix a été validé
});
```

PARTIE 5 — Questions de synthèse (3 pts)

Répondre brièvement :

1. Pourquoi Android utilise-t-il des Intents ?
2. Différence entre CheckBox et RadioButton ?
3. Quel est le rôle de la méthode `onCreate()` ?

PARTIE 6 — Spinner et ArrayAdapter

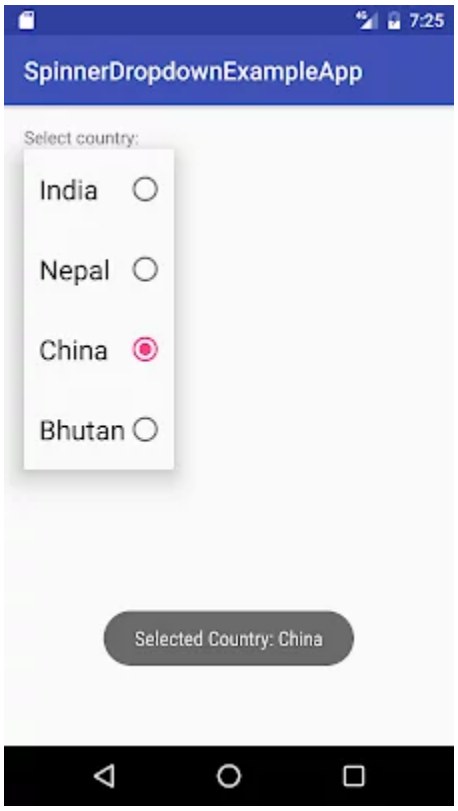


FIGURE 2 – Interface mobile contenant un Spinner pour la sélection d'un pays

Contexte

L'interface ci-dessus permet à l'utilisateur de sélectionner un pays à l'aide d'un Spinner.

6.1 Layout XML

```
<Spinner
    android:id="@+id/spCountry"
    android:layout_width="__________"
    android:layout_height="wrap_content" />
```

6.2 Code Java

Compléter le code permettant d'alimenter le Spinner :

```
Spinner spCountry;

spCountry = ______________________;

String[] countries = {
    "India",
    "Nepal",
    "China",
    "Bhutan"
};
```

```
ArrayAdapter<String> adapter =
    new ArrayAdapter<>(
        ------------,
        android.R.layout.simple_spinner_item,
        ------------
    );

adapter.setDropDownViewResource(
    android.R.layout.simple_spinner_dropdown_item
);

------------------------------;
```

6.3 Gestion de la sélection

Objectif :

Afficher le message suivant lors d'une sélection : **Selected Country : China**

```
spCountry.setOnItemSelectedListener(
    new AdapterView.OnItemSelectedListener() {

        @Override
        public void onItemSelected(
            AdapterView<?> parent,
            View view,
            int position,
            long id) {

            --------------------------------;
        }

        @Override
        public void onNothingSelected(
            AdapterView<?> parent) {
        }
    }
);
```

Questions

1. Quel est le rôle du `ArrayAdapter` ?
2. Quelle différence entre `Spinner` et `ListView` ?
3. Que représente le paramètre `position` ?